

Grundlagenpraktikum: Rechnerarchitektur

Secure Memory Unit

Team members:

Nour Marzouki

Mohamed Amine Chouchene

Omayma Yaich

Date: 19.08.2025

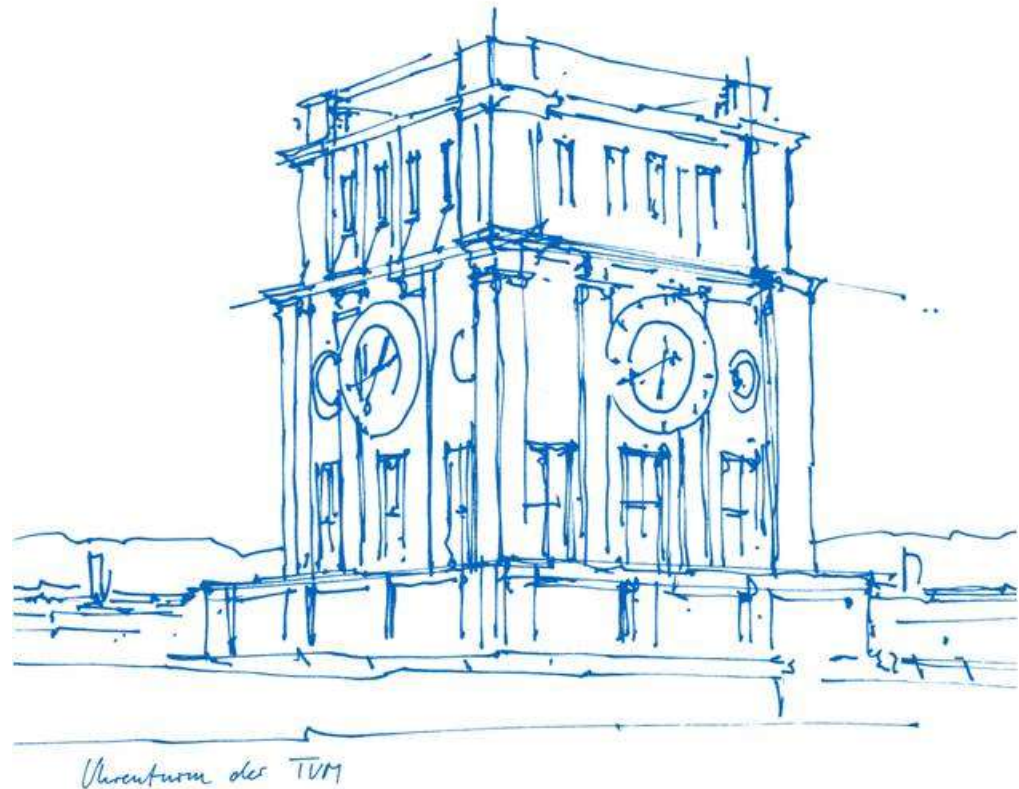


Table of contents

1. Problem Statement

2. Solution Approaches

3. Correctness

4. Evaluation

5. Summary and Outlook

Motivation: The Threat Landscape

- **Memory-based attacks increased by 1400% (2023)**, exploiting unencrypted data and weak access controls. [1]
- **Cold Boot Attacks:** Extract encryption keys from RAM post-shutdown by cooling chips. [2]
- **RowHammer:** Bit-flips in DDR4 RAM escalate privileges (e.g., Linux kernel hijacks [4]). [3]

=> Sensitive Data Exposure, Silent Data Corruption, Cloud Data Leaks (etc.. [5])

Possible Countermeasures

Threat	Countermeasure
Cold Boot Attacks	Encryption (XOR): Makes extracted RAM data useless without keys.
RowHammer / Bit-Flips	Parity Checks: Detects unintended bit-flips to preserve integrity.
Side-Channel Attacks (e.g., Spectre)	Address Scrambling: Prevents predictable access patterns.

Solution: Combining the proposed countermeasures in one Secure Memory Unit

Build a hardware-level security module that:

1. Uses **Xor-based symmetric encryption** (pRNG + XOR)
2. Hides access patterns (**Address Scrambling**)
3. Detects corruption (**Parity Checking**)

Table of contents

1. Problem Statement
- 2. Solution Approaches**
3. Correctness
4. Evaluation
5. Summary and Outlook

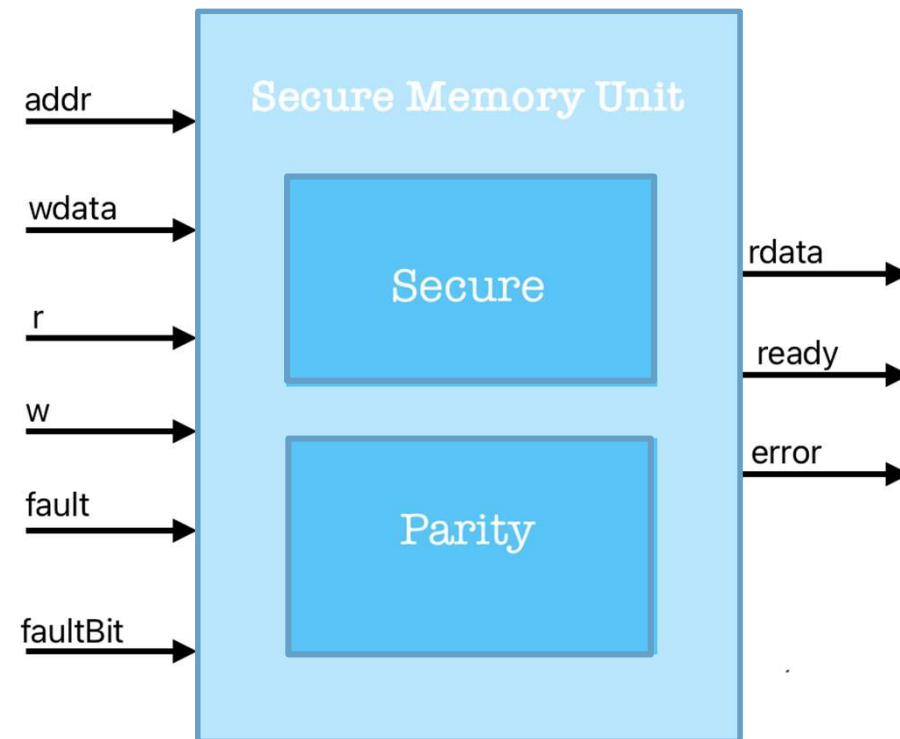
Our approach

SMU (Main Module):

Orchestrates memory access, fault injection, and module coordination.

Secure-Module: Handles encryption, decryption, and address scrambling.

Parity-Module: Dedicated to error detection (parity checks).



→ Modular structure promotes reusability, debugging, and future upgrades (e.g. better encryption)

Data Write Flow in Secure Memory

When writing 32-bit data, the system performs the following steps:

- 1  Divide input into 4 sub-words (bytes)
- 2  Generate subaddresses
- 3  Generate address scrambling key
- 4  Scramble address to obfuscate physical location
- 5  Create encryption key
- 6  Encrypt the data
- 7  Store data securely in memory
- 8  Calculate a parity bit for error detection
- 9  Save the parity in a separate map

Data Read Flow in Secure Memory

When reading 32-bit data, the system performs the following steps:

- 1  Retrieve Scrambling Key from Map
- 2  Generate subaddresses
- 3  Address-Scrambling
- 4  Retrieve data from memory
- 5  Retrieve decryption key
- 6  Data Decryption
- 7  Validate parity for error detection

Valid



Reassemble data words



Output 32-bit data

Invalid



Error

Fault Injection Flow in Secure Memory

Used to test system robustness by manually inverting specific data bits or the parity bit itself.

Data Bit Inversion

- 1  Retrieve data from memory
- 2  Invert Bit
- 3  Store modified data back in memory

Parity Bit Inversion

- 1  Access parity map
- 2  Invert Parity Bit

Integration of Secure Memory in TinyRISC CPU

We suggest **two** different approaches:

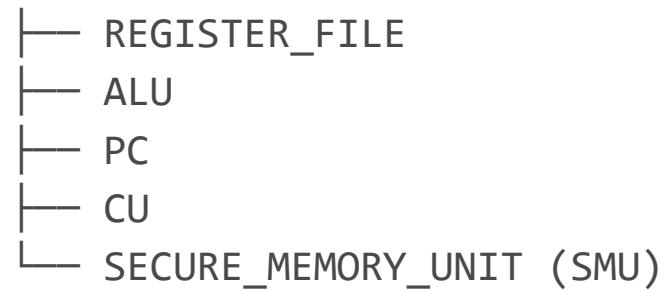
1. **Single Secure-Memory Architecture:** fully replace the main memory by the secure memory unit
2. **Dual Memory Architecture:** preserve the main memory and add a separate secure memory

Single Secure-Memory Architecture

Memory Replacement

Swap MEMORY → SECURE_MEMORY_UNIT

TinyRISC CPU



Single Secure-Memory Architecture:

Key Features

Preserve standard interface (r/w/addr/rdata)

Add:

- error output (1 = fault detected)
- fault input (32-bit injection address)
- faultBit input (4-bit flip selector)

Security Implementation

Data encryption via XOR

Address randomization

Parity error checking

CPU Adaptation

Pipeline control for variable latency (ready)

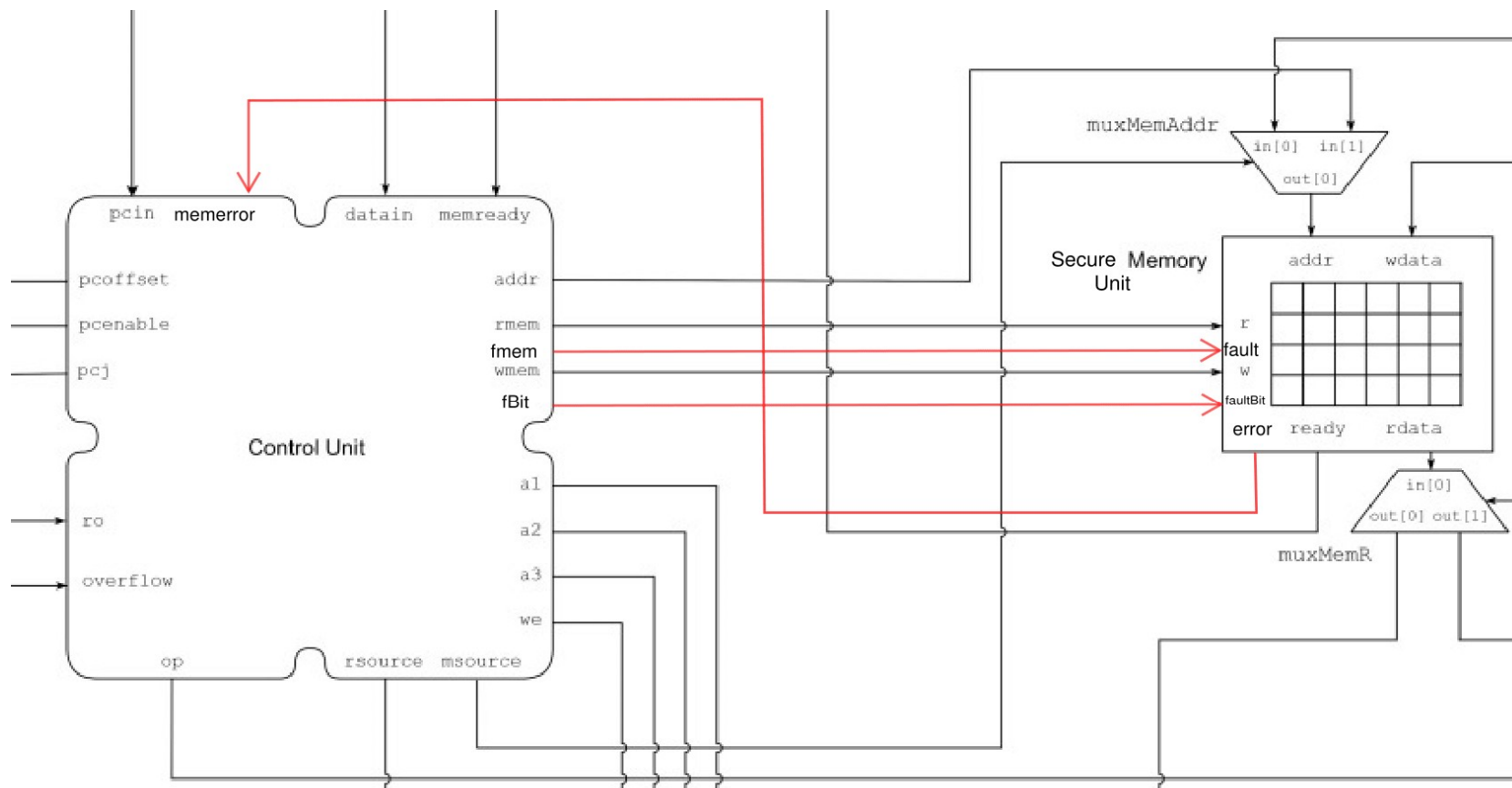
Fault response system (error)

Performance

Baseline +2-3 cycle penalty

Parallel processing optimization

Illustration



Dual-Memory Architecture:

Instead of replacing the main memory, our SMU can be added as a separate memory for secure operations. The CPU can then choose which memory to access based on **address range** ($0x00000000-0x0FFFFFFF \rightarrow \text{MAIN_MEMORY}$, $0x10000000-0x1FFFFFFF \rightarrow \text{SMU}$) or a **control signal** (`bool secure_access`)

TinyRISC CPU

├ REGISTER_FILE

├ ALU

├ PC

├ CU

├ MAIN_MEMORY (Standard memory for general data)

└ SECURE_MEMORY_UNIT (Encrypted, scrambled, parity-checked memory)

Dual-Memory Architecture: Memory Access Control

- **Standard Access (MAIN_MEMORY)**
 - Used for regular instructions/data (e.g., program code, non-sensitive data).
 - No encryption/scrambling, minimal latency

=> Fast but unsafe
- **Secure Access (SMU)**
 - Used for sensitive data (e.g., cryptographic keys, passwords).
 - Encrypted, address-scrambled, and parity-checked.

=> Safe but slower

How safe are the used security mechanisms

Mechanism	Purpose	Weakness
LCG (pRNG)	Key generation	Not cryptographically secure => Attackers can predict future values with few known outputs.
XOR Encryption	Data confidentiality	Predictable with weak keys (32-bit Key Space can be vulnerable to Brute-Force Attacks)
Address Scrambling	Preventing side-channel	Key recovery via pattern analysis
Parity Bit	Error detection	- Even number of corrupted Bits remains undetected. [6] - The location of the corrupted Bit remains unknown and thus uncorrected

Can Parity Detect All Errors?

Here's a clear example illustrating how a parity bit fails to detect a **two-bit error**:

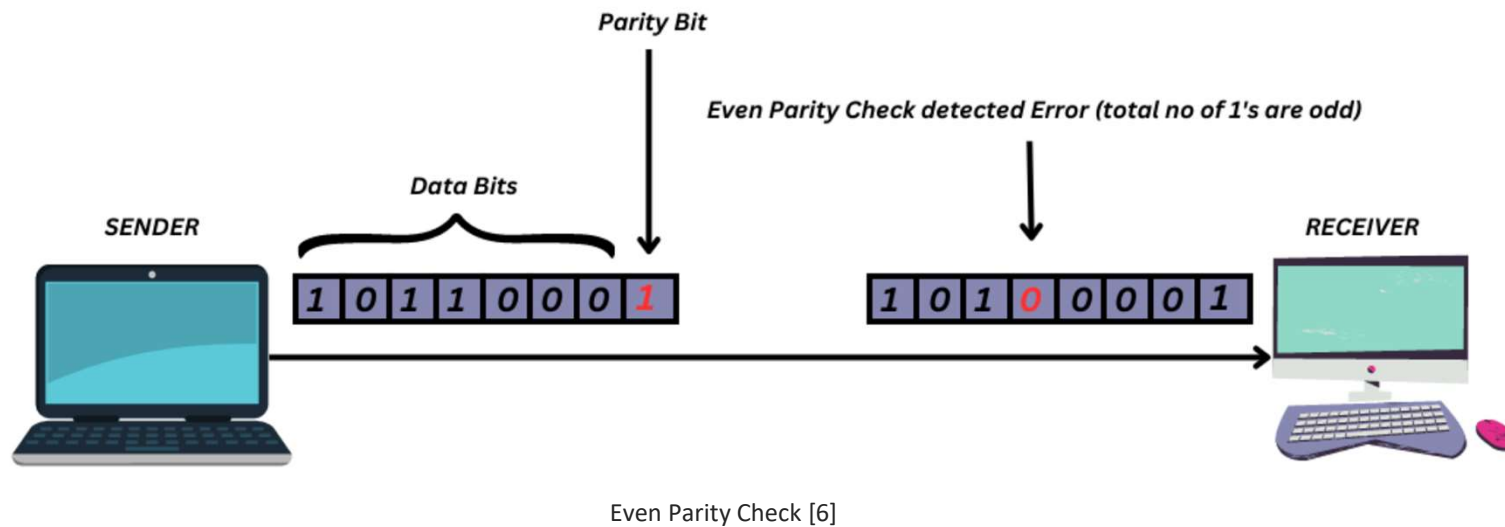


Table of contents

1. Problem Statement
2. Solution Approaches
- 3. Correctness**
4. Evaluation
5. Summary and Outlook

Correctness

1. Simple Test:

A test scenario was executed in which a write operation was performed to address 0x1000 (4096) with the value 2882343476. The data was successfully scrambled, encrypted, and stored. Upon reading from the same address, the data was correctly decrypted and descrambled, confirming the integrity of the secure memory operations.

```
=== [START READ/WRITE] ===  
[WRITE] Writing to address 4096  
1 ns: [PRNG] New PRNG value = 889114580  
[KEYGEN] New data key for address 4096 = 212  
1 ns: [ENCRYPT] Encrypting 2882343476 with key 212  
  [ENCRYPT] Byte[0] = 52 XOR 212 = 224  
  [ENCRYPT] Byte[1] = 18 XOR 212 = 198  
  [ENCRYPT] Byte[2] = 205 XOR 212 = 25  
  [ENCRYPT] Byte[3] = 171 XOR 212 = 127  
[MEMORY][WRITE] Addr: 3067923977 = 224 (Byte[0])  
[MEMORY][WRITE] Addr: 3067923976 = 198 (Byte[1])  
[MEMORY][WRITE] Addr: 3067923979 = 25 (Byte[2])  
[MEMORY][WRITE] Addr: 3067923978 = 127 (Byte[3])  
[PARITY][WRITE] Checking byte 0 @ addr 3067923977  
[PARITY][WRITE] Checking byte 1 @ addr 3067923976  
[PARITY][WRITE] Checking byte 2 @ addr 3067923979  
[PARITY][WRITE] Checking byte 3 @ addr 3067923978  
[Write] Done
```



```
=== [START READ/WRITE] ===  
[READ] Reading from address 4096  
  [DECRYPT] Byte[0] = 224 XOR 212 = 52  
  [DECRYPT] Byte[1] = 198 XOR 212 = 18  
  [DECRYPT] Byte[2] = 25 XOR 212 = 205  
  [DECRYPT] Byte[3] = 127 XOR 212 = 171  
[PARITY][READ] Byte[0] = 52 @ 3067923977  
[PARITY][READ] Byte[1] = 18 @ 3067923976  
[PARITY][READ] Byte[2] = 205 @ 3067923979  
[PARITY][READ] Byte[3] = 171 @ 3067923978  
[READ RESULT] rdata = 2882343476  
[Read] Done
```

Write > Inject > Read

[illegible]

Correctness

2. Secure Memory Extreme Validation:

This test executed 200,000 memory operations (100,000 writes and 100,000 reads) across approximately 5.7 million cycles. No faults were injected during the run, and 0 errors were detected.

```
=== Secure Memory I/O Analysis ===  
Total cycles: 5700000  
Operations completed: 200000  
  Reads: 100000  
  Writes: 100000  
Total faults injected: 0  
Errors detected: 0  
  
Latency Distribution:  
  Encryption: 600000 (37%)  
  Scrambling: 200000 (12%)  
  Memory Access: 800000 (50%)  
  
Simulation Results:  
  Total Cycles: 5700000  
  Errors Detected: 0
```

Table of contents

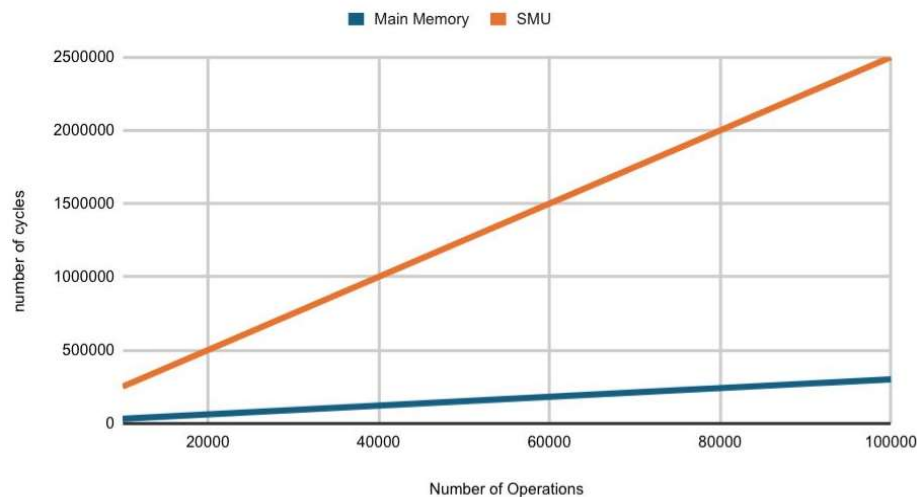
1. Problem Statement
2. Solution Approaches
3. Correctness
- 4. Evaluation**
5. Summary and Outlook

Secure vs. Main Memory Performance Tradeoff: By the Numbers

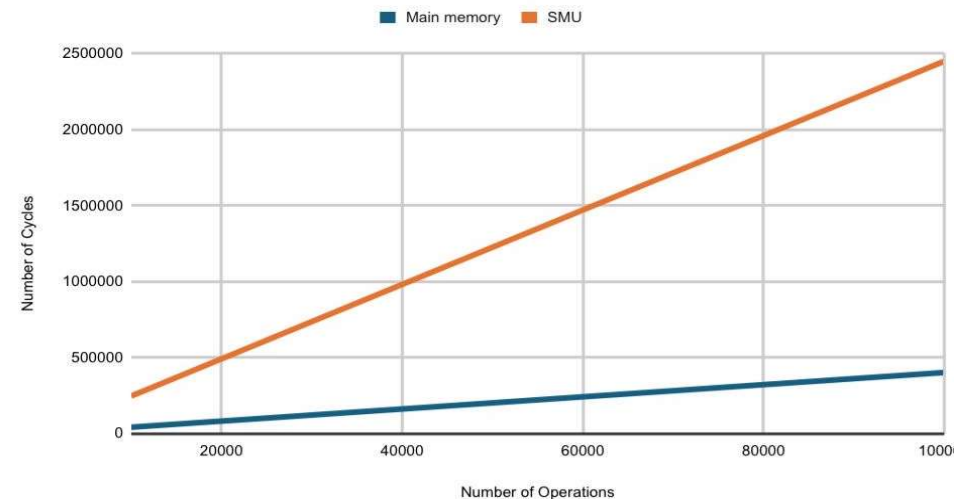
Approach: we compared the performance of our secure memory unit to the memory unit proposed by the homework in different overloads:

Results:

Performance of Main Memory and SMU in a read only workload

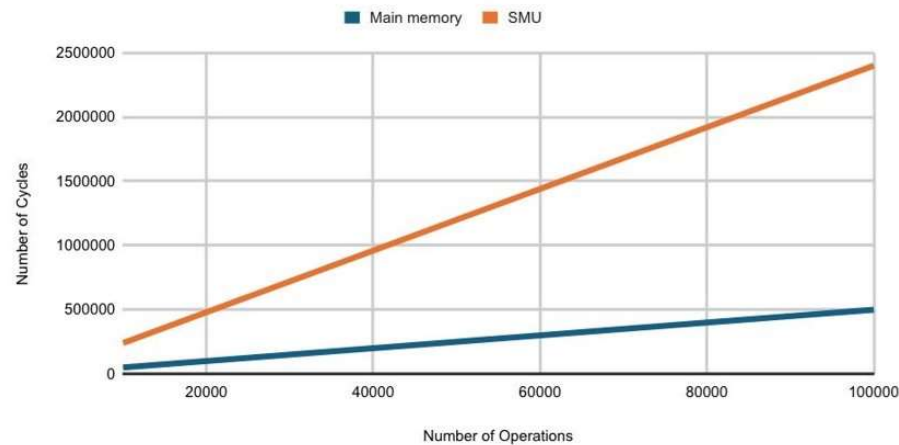


Performance of Main Memory and SMU in a mixed workload (50%Read 50% Write)

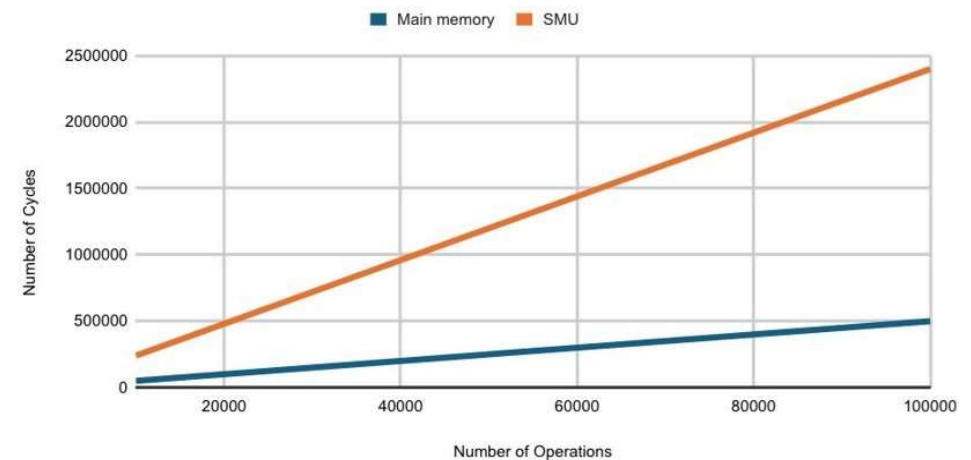


Results:

Performance of Main Memory and SMU in a write only workload



Performance of Main Memory and SMU in a write only workload



Deductions:

From the graphs, we can clearly define the latencies for each operation and each memory, thus deducing the penalty :

Memory type	Main Memory	SMU	Overhead (MMU cycles/ SMU cycles)
Number of cycles/read op	3	25	8.3
Number of cycles/write op	5	24	4.8

Security vs. Performance: Design Implications

- ❖ Interestingly , the smu requires fewer cycles for writes than reads . This is due to its use of **parallel encryption and scrambling**, during write operations, which reduces internal processing delays.
- ❖ These latency overheads are not design flaws, but **intentional tradeoffs** :
->Each operation through the SMU is **cryptographically hardened**, making it resistant to data leakage, tampering, or unauthorized access. That said, this protection comes at a **significant compute and latency cost**.

=> Strategic Recommendation

(+)Use SMU selectively for:

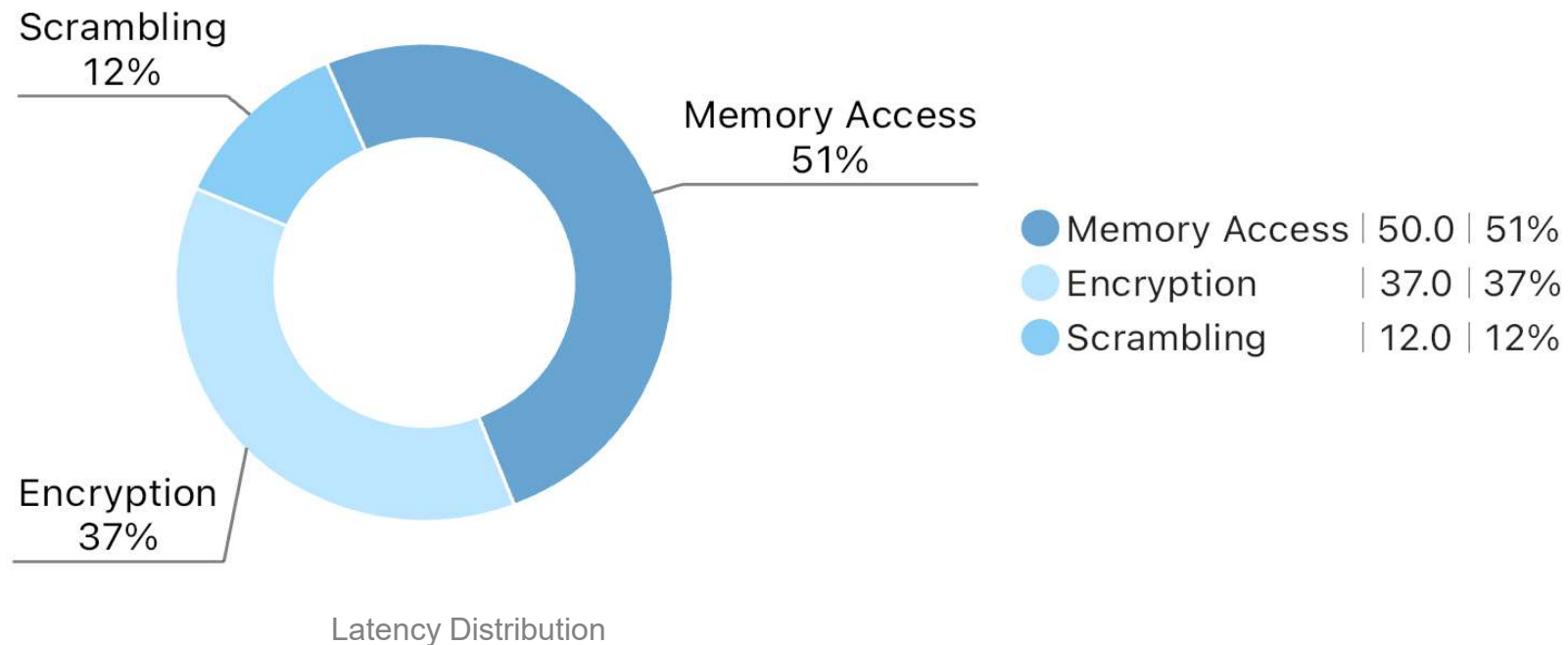
- ❖ Cryptographic keys
- ❖ Authentication tokens
- ❖ Trusted data

(-)Avoid SMU for:

- ❖ Bulk I/O
- ❖ Temporary buffers
- ❖ Performance-critical but non-sensitive data

Identifying Performance Bottlenecks in the SMU

From the latency distribution, we observe the following breakdown (example: 10000 operations for default latencies):



=> This reveals that **memory access is the dominant bottleneck**, accounting for **half of the total latency**.

*Why Memory Access Becomes a Bottleneck

- > Real memory systems are limited by access speed (e.g., DRAM or cache latency).
- > Our Secure Memory Unit already employs **parallel encryption and scrambling**, reducing internal processing delays. However, the primary bottleneck remains **memory access latency** — just like in real-world secure systems.

=> Mitigation Strategies: inspired from Real-World Architectures:

- ❖ Hardware Acceleration
- ❖ Prefetching & Caching
- ❖ Multi-bank Memory Design

Table of contents

1. Problem Statement
2. Solution Approaches
3. Correctness
4. Evaluation
- 5. Summary and Outlook**

Summary

Our Secure Memory Unit provides a proof-of-concept hardware implementation that mitigates key vulnerabilities in modern memory systems. By combining **XOR-based encryption**, **address scrambling**, and **parity checking**, it offers improved data confidentiality and integrity.

While these mechanisms significantly strengthen memory protection, they are **not sufficient against sophisticated or large-scale attacks**.

Performance measurements show moderate overhead, with memory access latency being the dominant bottleneck.

Looking ahead, integrating more robust cryptographic algorithms and advanced error correction techniques could further enhance the resilience of our secure memory system.

Potential Improvements

- **Use Cryptographically Secure PRNGs (CSPRNGs):**
Replace LCG with secure PRNGs like AES-CTR, ChaCha20..
- **Adopt Stronger Encryption:**
Implement lightweight block ciphers (e.g., SPECK, SIMON) that are harder to crack than outdated ciphers.
- **Extend Key Lengths:**
A 32-bit key (2^{32} combinations) can be cracked in seconds on a laptop.
=> Move from 32-bit to 128-bit or 256-bit keys to resist brute-force.
- **Upgrade Error Detection/Correction:**
Relying solely on parity leaves the system vulnerable to real-world threats. For stronger protection:
 - **Hamming Code:** Corrects 1-bit, detects 2-bit errors [7]
 - **ECC (Error-Correcting Code):** Used in RAM for bit-flip resilience [8]
 - **Reed-Solomon:** Ideal for burst-error correction [9]

Sources:

- [1]: <https://digests.digitalisationworld.com/news/65793/1400-surge-in-memory-based-attacks>
- [2]: <https://es.cs.rptu.de/publications/datarsg/Bitt24.pdf>
- [3]: <https://www.twingate.com/blog/glossary/row%20hammer>
- [4]: <https://googleprojectzero.blogspot.com/2015/03/exploiting-dram-rowhammer-bug-to-gain.html>
- [5]: <https://secureframe.com/blog/data-breach-statistics>
- [6]: <https://www.geeksforgeeks.org/digital-logic/error-detection-codes-parity-bit-method/>
- [7]: <https://www.geeksforgeeks.org/computer-networks/hamming-code-in-computer-network/>
- [8]: https://www.vpnunlimited.com/de/help/cybersecurity/error-correcting-code?srsItd=AfmBOopoQrCpqriUUg7tsdn-iTRKkjxjH_f_u5fhclF3bOmD8FKGwRrZ
- [9]: https://tu-dresden.de/ing/elektrotechnik/ifn/itml/ressourcen/dateien/lehre/vl/codth/A2_RSC.pdf?lang=en

Grundlagenpraktikum: Rechnerarchitektur

Secure Memory Unit

Thank You for your attention!

Team75 Members

